

Guión de grabación — Clase 2

La VPS: aprovisionar, entrar y cerrar

Programación 3 · UTN FRM · Matías Santiago Torres — 29 diapositivas · 3 videos · 15 ejemplos

Cuánto dura, y por qué son siete minutos y no seis

Los tiempos están calculados a **165 palabras por minuto**, el mismo ritmo con el que se midió la Clase 1. A esa velocidad los tres videos dan **6:57 · 7:21 · 7:51**, total 22:09. La columna de 145 está para el ritmo pausado: tres minutos más sobre el total.

Este capítulo no entra en tres videos de seis minutos, y conviene saber por qué antes de intentarlo. La Clase 1 tenía un solo tema —la resolución de nombres— contado en treinta diapositivas. Esta tiene cuatro que no se deducen entre sí: virtualización, SSH, el modelo de red del núcleo, y el problema de Docker. Cada uno arranca de cero. Achicarlos a seis minutos significaba sacar la teoría de virtualización o la anatomía de netfilter, y sin netfilter el video 3 no se puede explicar: se convierte en "acordate de esto porque sí".

La palanca para acortar son los pasajes con ✂ tijera, que van en gris. Sacándolos todos, los tres bajan a **6:21 · 6:58 · 7:19** y el total a 20:39. **Ninguno contiene algo que se evalúe ni que haga falta para entender lo que sigue** — y ninguno es un ejemplo.

Los timecodes de cada diapositiva son acumulados y salen del conteo real de palabras, no de una estimación. Sirven para corregir el rumbo *durante* la grabación: si a la diapositiva 15 vas dos minutos arriba, no lo arreglás en las últimas cuatro.

Lo **resaltado en amarillo** es lo único que no conviene parafrasear: son las frases que después vuelven como criterio de corrección.

Los quince bloques de EJEMPLO: cómo se dicen

Van en verde y valen la misma regla que en la Clase 1: **no los leas, contalos**. Son el único lugar del guión donde conviene despegar la vista del texto y mirar a cámara — el alumno tiene que notar que cambiaste de registro.

Cada uno **reemplaza** una explicación abstracta que saqué del guión, no se le suma encima. Si te sobra tiempo, el ejemplo no es lo que se corta: cortándolo volvéis a la versión aburrida y encima incompleta.

El reparto es desparejo a propósito: **siete en el video 1 y seis en el 2**, que son los que tienen la abstracción pesada, y **dos en el 3**, que ya es concreto de por sí — ahí el ejemplo es el propio panel de Easypanel expuesto en el 3000.

⚠ Lo que el video no hace, y hay que decirlo en clase

En veintidós minutos no entra el capítulo: entra el hilo del capítulo. El manuscrito de la Clase 2 tiene más de diez mil palabras; este guión tiene tres mil seiscientas. El video sirve para que el alumno sepa *qué* tiene que entender y *por qué* importa; el apunte es el que lo explica.

Las tablas de riesgos por puerto, de endurecimiento y de errores frecuentes **se muestran, no se narran**: son de consulta y están en el apunte.

⚠ Antes de apretar grabar

No leas la diapositiva. El alumno la está viendo. El guión está escrito para *decir lo que la diapositiva no dice*: por qué está ahí, qué error previene, con qué se conecta.

Grabá los tres por separado, no uno largo cortado. Cada uno abre y cierra solo: si cambia el panel de Hostinger, regrabás el video 1 y los otros dos siguen sirviendo. Nombres sugeridos: C2-V1-vps-y-claves, C2-V2-puertos-y-firewall, C2-V3-docker-y-cierre.

Este capítulo tiene dos diapositivas que no se pueden apurar: la 15, que es el recorrido del paquete, y la 17, que es el problema de Docker. Todo lo demás se puede contar más rápido; esas dos son la única razón por la que este video existe y no alcanza con leer un tutorial.

Video	Título	Diapos	Ejem.	Palabras	A 165 p/min		A 145 p/min
					completo	sin tijeras	
1	La máquina prestada y la llave propia	1 – 10	7	1147	6:57	6:21	7:55
2	Puertos, sockets y el firewall de verdad	11 – 16	6	1213	7:21	6:58	8:22
3	Docker se saltea el firewall, y cómo se cierra el servidor	17 – 29	2	1294	7:51	7:19	8:55
Total			15	3654	22:09	20:39	25:12

✦ Por qué el corte cae ahí, y por qué el reparto es desparejo

Cada video cierra con una pregunta que el siguiente contesta. El 1 termina en "*además de ustedes, ¿quién más puede entrar?*"; el 2 responde con los puertos y el firewall, y cierra en "*eso es lo que ufw dice; en el video tres vamos a ver si es verdad*"; el 3 abre demoliendo esa afirmación. **Si reordenás los bloques se pierde el enganche**, y el 3 pierde además su motivo.

El video 2 tiene seis diapositivas y dura más que el 1, que tiene diez. No es un error de reparto: son las seis diapositivas más densas del capítulo —tupla, interfaces, rangos, netfilter, el recorrido del paquete y ufw— y son las que sostienen todo el video 3. El 3, al revés, tiene trece diapositivas porque las últimas cuatro son de consulta y se muestran en pocos segundos.

Video 1 — La máquina prestada y la llave propia

Diapositivas 1 a 10 · 1147 palabras · 6:57 (sin tijeras 6:21)

1

0:00

Portada

0:00 – 0:14

Capítulo dos. La clase pasada consiguieron un nombre; hoy consiguen la máquina a la que ese nombre apunta.

El título tiene tres verbos: aprovisionar, entrar y cerrar. **El primero lleva cinco minutos; los otros dos son la clase.**

Cuatro bloques, y fíjense en la proporción: uno es crear el servidor, tres son no perderlo.

La frase del capítulo: **un servidor recién creado es, por defecto, un objetivo.** No cuando tenga usuarios. Desde el minuto cero, con la calculadora vacía adentro.

Al terminar: VPS operativo, entrada por claves, firewall puesto y panel con candado.

Un VPS no es un tipo de computadora. Es una ilusión, y está construida a propósito.

EJEMPLO

Reciben lo que parece una máquina entera —su sistema operativo, su IP, su disco— y están compartiendo el hierro con treinta desconocidos. Es un edificio de departamentos: la llave es tuya, las paredes son de verdad, y la bomba de agua es de todos.

✂ Arrancó por plata. Años sesenta: una computadora central costaba una fortuna y corría un programa por vez. Máquina ociosa es dinero quemado. IBM lo atacó con VM/370 en el 72.

La diferencia con el tiempo compartido es la que importa: **ahí los usuarios conviven adentro de un mismo sistema operativo y se ven; en virtualización cada uno tiene el suyo y no sabe que los demás existen.**

1974. Dos tipos definen qué tiene derecho a llamarse virtualizador y les salen tres condiciones que siguen valiendo. **Equivalencia:** el huésped no se da cuenta de que está adentro. **Control:** el hipervisor tiene la última palabra. **Eficiencia:** las instrucciones corren directo sobre el procesador, sin emular.

Y acá el dato que explica treinta años de historia: **x86 no cumplía la condición.**

EJEMPLO

Por eso esto existía en los mainframes desde el 72 y recién llegó a los servidores comunes en 2006: primero traducción en vivo con VMware, después Xen modificando el huésped, y finalmente Intel y AMD lo metieron adentro del procesador. Eso último es lo que hace que la VPS de ustedes rinda casi como una máquina real y no como un emulador.

✂ La tabla es de ubicación: tipo 1 corre sobre el hierro, tipo 2 adentro de un sistema operativo. El VirtualBox que usaron es tipo 2; la VPS de ustedes es tipo 1, casi seguro KVM.

Bajen a las tres cajas, que es lo que van a ver con los ojos este cuatrimestre.

EJEMPLO

Uno: el aislamiento es real, el hierro se comparte. Los proveedores venden más núcleos virtuales que reales apostando a que no todos estarán al mango a la vez. Cuando la apuesta falla aparece el vecino ruidoso: el servidor se pone lento un martes y ustedes no tocaron nada. No están locos, y no es su código.

Dos: el estado entero de la máquina es un archivo. Por eso reinstalar tarda tres minutos, y por eso **esa operación borra todo y no tiene deshacer.**

Tres, la que más les va a doler: la responsabilidad se corta en un punto preciso. El proveedor responde por hipervisor, hardware y red. **De la puerta del sistema operativo para adentro es de ustedes,** y no es una postura comercial: el proveedor, por diseño, no puede ver adentro.

Cuatro filas, un solo eje: **cuánta abstracción hay entre el que programa y la máquina.** Cada escalón para arriba te saca trabajo y te saca, en la misma cantidad, visibilidad.

Y me lo van a preguntar: si Vercel resuelve el deploy en tres clics, ¿por qué una VPS?

EJEMPLO

Justamente por eso. Un PaaS no te deja ver la tabla de reglas del firewall, y no porque te la escondan: del lado del cliente esa tabla no existe. Las piezas están —alguien las configuró—, pero no se observan.

El PaaS resolvería el despliegue en menos pasos y les ocultaría todo lo que la materia quiere enseñar: red, puertos, proxy inverso, certificados, aislamiento. **Lo que no se observa no se aprende.**

Esta es la única diapositiva donde se crea el servidor: cuatro pasos en hPanel, plantilla Easypanel sobre Ubuntu 24.04, y a esperar.

Lo que quiero que entiendan es *qué* hace ese botón: escribe una imagen de disco ya preparada encima de la que había. Por eso tarda minutos —es copiar un archivo— y por eso **borra todo lo anterior sin deshacer.**

✂ Al terminar anoten la IP pública y verifiquen que sea la misma que cargaron en el DNS la clase pasada.

SSH nació de un robo. Hasta mediados de los noventa, Telnet y rlogin mandaban **la contraseña en texto plano**. A Tatu Ylönen le capturaron las credenciales en la red de su universidad y en 1995 escribió la primera versión.

Las tres capas van en ese orden y el orden dice algo: la de transporte cifra el canal y **autentica al servidor ante el cliente, no al revés.** Recién sobre ese canal seguro el cliente demuestra quién es.

EJEMPLO

De ahí sale el mensaje que todos ven y nadie lee. La primera vez el cliente no puede saber si esa clave de anfitrión es la legítima: pregunta, ustedes aceptan, y la guarda. Es vulnerable exactamente una vez —la primera— y desde ahí detecta cualquier suplantación. Por eso, si les aparece el cartel grande de que la clave cambió, no es un trámite: o reinstalaron el servidor, o hay alguien en el medio.

Cuatro filas y ninguna habla de comodidad: todas hablan de seguridad.

Saquen de la cabeza la frase "se copia la clave al servidor", porque induce a error. Esto es un **desafío criptográfico**: el servidor manda un dato aleatorio, el cliente lo firma con la privada, el servidor verifica la firma con la pública. **La clave privada nunca se transmite, ni siquiera cifrada.**

EJEMPLO

Miren la fila tres, que es la que más rinde. Si mañana les roban el servidor entero, el atacante se lleva las claves públicas de los cuatro y no le sirven para nada: con una pública se verifica una firma, no se firma. Es robarse la cerradura creyendo que es la llave.

El comando va **en la máquina de ustedes**, nunca en el servidor. La que termina en punto pub se comparte; la otra no sale de su computadora jamás.

✂ Hay bots que escanean GitHub buscando claves privadas subidas por error, y tardan minutos.

Cada integrante instala **su propia** clave: cuatro accesos independientes, revocables por separado, sin contraseña compartida.

EJEMPLO

Y no es comodidad, es auditoría. Una contraseña que saben cuatro es una identidad sin dueño: cuando pasa algo, entró "el grupo". Con claves, cada línea tiene nombre y apellido, y sacar a uno es borrar un renglón.

Después se cierra la contraseña. Ojo con la segunda directiva: `prohibit-password` no prohíbe entrar como root, prohíbe entrar como root **con contraseña**.

Verifiquen que las cuatro claves andan ANTES de tocar ese archivo. Segunda terminal, entran con la clave, y recién ahí editan. Si cierran la única sesión que tenían, quedaron afuera de su propio servidor.

Fin del video 1. Cierre sugerido a cámara: *"Ya tienen la máquina y ya tienen la llave. Falta la parte incómoda: además de ustedes, ¿quién más puede entrar? Eso es el video dos."*

Video 2 — Puertos, sockets y el firewall de verdad

Diapositivas 11 a 16 · 1213 palabras · 7:21 (sin tijeras 6:58)

11

0:00

Puertos y sockets: qué significa "escuchar"

0:00 – 1:25

Video dos. Terminamos con quién puede entrar; ahora vamos a **por dónde**.

La definición corta la saben: la IP identifica una máquina, el puerto identifica un servicio adentro de esa máquina. Pero la formulación precisa es más útil, porque explica cosas que si no parecen magia. **Una conexión establecida no se identifica por un puerto: se identifica por una tupla de cuatro elementos** — IP de origen, puerto de origen, IP de destino, puerto de destino—.

EJEMPLO

¿Cómo hace un servidor con un solo puerto 443 para atender diez mil conexiones a la vez sin mezclarlas? No hay diez mil puertos: hay diez mil tuplas distintas. Es la centralita de una empresa: un solo número de teléfono, y adentro cien conversaciones que no se pisan porque cada una tiene un llamante distinto.

Y "escuchar" es una orden concreta al sistema operativo: *entregame a mí toda conexión nueva dirigida a este número*.

Ahora la tabla de la derecha, que parece un detalle de configuración y es una decisión de seguridad de primer orden: un proceso además declara **en qué interfaz** escucha.

EJEMPLO

Un servicio que escucha en 127.0.0.1 **es inalcanzable desde internet aunque el firewall esté apagado y el puerto abierto**. No hay firewall que valga: simplemente no hay nadie atendiendo del lado de afuera. Es la forma más barata y más robusta de cerrar algo, y vuelve en la Clase 3 y en la Clase 5.

12

1:25

Rangos de puertos e inspección del estado

1:25 – 2:45

Sesenta y cinco mil quinientos treinta y cinco puertos, y no son todos iguales. La IANA los parte en tres rangos, normados en la RFC 6335.

✂ Bien conocidos hasta el 1023, asignados por trámite formal; registrados hasta el 49151, trámite simplificado; y de ahí para arriba, efímeros: no los asigna nadie, los toma el sistema operativo.

De ahí salen dos consecuencias prácticas. La primera está abajo en negrita: **en Unix, escuchar por debajo del 1024 requiere privilegios de administrador**.

EJEMPLO

Es una restricción histórica para que un usuario cualquiera no pudiera hacerse pasar por el servidor web de la máquina. Y es exactamente la razón de algo que van a ver la clase que viene: el backend de ustedes escucha en 3000, o en 8000, y hay un proxy inverso adelante ocupando el 80 y el 443. No es una convención estética, es un permiso que el proceso no tiene.

La segunda: **la asignación es una convención, no una imposición técnica**. Nada impide correr SSH en el 2222; lo que se pierde es que las herramientas lo encuentren solas.

Y el comando de la derecha es el que van a correr en el servidor. Las cuatro banderas importan, pero la que hace el trabajo conceptual es la `-l`: pide los sockets en estado LISTEN, los que esperan conexiones nuevas.

Esta es la salida real de un servidor recién aprovisionado. Cuatro procesos escuchando: sshd en el 22, Traefik en el 80 y el 443, Easypanel en el 3000.

Y la frase del medio es la que quiero que se lleven: **cada puerto abierto es un proceso concreto que alguien decidió levantar, no una propiedad mágica del servidor.** No hay puertos que "vienen abiertos": hay programas corriendo.

Ahora la pregunta de abajo, y les pido que la contesten antes de que la conteste yo. De estos cuatro, ¿cuáles necesita ver el mundo entero?

 **Dirección:** dejá dos segundos de silencio real acá. Es la única pregunta del capítulo que se contesta sola si la piensan.

El 22 lo necesitan ustedes cuatro, no el mundo. El 3000 no lo necesita nadie desde afuera, y vamos a ver por qué es peligroso. **La respuesta es 80 y 443, y es toda la clase de seguridad resumida en dos números.**

✂ Corran después el mismo comando sin la `-l` y miren cuánto crece la salida: esa diferencia son las conexiones ya establecidas. El socket en escucha es uno, las conversaciones en curso son muchas.

Primera corrección, de vocabulario: **el firewall no es ufw.** En Linux el firewall es un subsistema del núcleo llamado netfilter, y iptables, nft y ufw no son el firewall: son formas de escribirle reglas. Parece una distinción de purista, y es la causa exacta del problema del video tres.

Netfilter define cinco **puntos de enganche** en lugares precisos del recorrido que hace un paquete adentro del núcleo. En cada punto se evalúan las reglas que ahí haya registradas.

EJEMPLO

Pensalo como una aduana con cinco mostradores fijos en el pasillo. El paquete no elige por cuál pasa: el recorrido está marcado. Y si vos pusiste a tu inspector en el mostrador tres, el paquete que va por el pasillo de al lado nunca lo ve. No es que el inspector falle: es que no pasa por ahí.

La pieza decisiva está entre PREROUTING y los dos siguientes: **la decisión de enrutamiento.** Después de PREROUTING el núcleo mira la dirección de destino y se pregunta si el paquete es para él o para otro. Si es para él, sigue por INPUT. Si es para otro, sigue por FORWARD.

Son dos caminos excluyentes: un paquete pasa por uno o por el otro, nunca por los dos. Subrayen esa frase; en veinte minutos les explica todo.

Esta figura es el corazón del capítulo. Siguen la bifurcación con el dedo mientras hablo.

El paquete entra, pasa por PREROUTING, y ahí está el rombo: ¿es para esta máquina o para otra? Para esta máquina, INPUT. Para otra, FORWARD.

Y ahora la línea que hay que memorizar, que está escrita arriba: **ufw escribe sus reglas en INPUT; el tráfico hacia los contenedores de Docker pasa por FORWARD.**

Léanla dos veces. Todavía no dije cuál es el problema y ya lo tienen adelante.

Las tres cajas de abajo son la otra clasificación: netfilter agrupa las reglas en tablas según para qué sirven. **filter** acepta o descarta, y ahí viven las reglas de ufw. **nat** reescribe direcciones, y ahí vive Docker. **mangle** modifica campos y no lo vamos a tocar.

EJEMPLO

Esa tabla nat es la que convierte "el puerto 5432 de mi servidor" en "el puerto 5432 de aquel contenedor", que tiene otra IP. Es un cambio de sobre: llega dirigido a una dirección y sale dirigido a otra, antes de que nadie decida por qué pasillo va.

Con el modelo puesto, ufw se vuelve trivial: es una capa de conveniencia que traduce órdenes legibles a reglas de netfilter.

Miren el orden de los seis comandos, porque el orden **es** el contenido. Los dos primeros son la política:

descartar todo lo que entra y permitir todo lo que sale. Eso no es paranoia, es el principio de valores predeterminados seguros que vemos al final del capítulo: lo que no está explícitamente permitido, se niega.

Recién después se abren tres puertos. El 22 porque sin eso pierden el acceso; el 443 porque es la aplicación. Y el 80 —esto lo preguntan todos los años— **no se puede cerrar aunque todo vaya por HTTPS**: Let's Encrypt valida la titularidad del dominio pidiendo un archivo por el puerto 80. Sin 80 no hay certificado, y sin certificado no hay HTTPS.

EJEMPLO

Y la caja de abajo no es un consejo, es una advertencia con víctimas. Si corren `ufw enable` antes de permitir el 22, el firewall les corta la propia sesión SSH en el acto: están serruchando la rama en la que están sentados. Se recupera por la consola de emergencia de Hostinger, pero es media clase perdida. Permitir primero, habilitar después. Siempre.

Fin del video 2. Cierre sugerido a cámara: "Con esto, el servidor tiene tres puertos abiertos y todo lo demás cerrado. Eso es lo que ufw dice. En el video tres vamos a ver si es verdad."

Video 3 — Docker se saltea el firewall, y cómo se cierra el servidor

Diapositivas 17 a 29 · 1294 palabras · 7:51 (sin tijeras 7:19)

17

0:00

El problema: Docker no respeta el firewall

0:00 – 1:07

Video tres. Esta es la diapositiva más importante del capítulo y la que menos aparece en los tutoriales de despliegue. Siguen las tres cajas en orden.

Primera: un contenedor **no comparte la pila de red del anfitrión**: tiene su propia interfaz y su propia IP privada. **Para el núcleo, un contenedor es otra máquina.**

Segunda, y ya la pueden anticipar: el paquete entra por PREROUTING, donde Docker le reescribe la dirección de destino; llega la decisión de enrutamiento, el núcleo mira el destino ya reescrito y concluye que no es para él; y lo manda por FORWARD. **Nunca pasa por INPUT. Y las reglas de ufw viven en INPUT.**

Tercera, la consecuencia: corren `ufw deny 5432`, levantan el contenedor con `-p 5432:5432`, y la base queda accesible desde internet mientras `ufw status` informa que está bloqueada. Las dos cosas son ciertas: `ufw` escribió la regla, y esa regla no se evalúa nunca para ese tráfico.

Y no es un error de Docker ni una mala configuración de Ubuntu: son dos programas escribiendo reglas en el mismo subsistema, en puntos distintos, sin conocerse.

18

1:07

Verificación desde el exterior

1:07 – 1:59

Entonces, ¿cómo sé qué tengo abierto? Desde adentro, no. **La única comprobación confiable es consultar los puertos desde otra máquina**, porque cualquier herramienta que corra adentro describe lo que ella cree, y solo una prueba desde afuera atraviesa el camino que atravesaría un atacante.

EJEMPLO

Y fíjense en la forma del error, porque se repite en toda la ingeniería: **una herramienta te informó sobre sí misma y vos lo leíste como información sobre el sistema.** `ufw status` no dice "el 5432 está cerrado"; dice "yo tengo una regla que lo cierra". Un montón de bases expuestas en todo el mundo están así porque alguien corrió `ufw deny` y se quedó tranquilo.

✂ La bandera `-Pn` le dice a `nmap` que no pregunte primero con un ping si el anfitrión está vivo: muchos proveedores los descartan, y sin eso `nmap` ni siquiera escanea.

19

1:59

Cómo se evita el problema

1:59 – 2:37

Tres estrategias, en orden de preferencia y no de dificultad.

La primera es la que usamos, y es la más aburrida: **no publicar el puerto**. El que se pregunte cómo hace entonces la API para hablarle a la base, la respuesta es la red interna de Docker, que es el contenido central de la Clase 5.

La segunda es lo que vimos con las interfaces: publicar en 127.0.0.1 y llegar por un túnel SSH.

Y la frase de abajo conecta el capítulo entero: **la red interna no es una optimización, es la solución a este problema. El puerto más seguro es el que no existe.**

Acá viene la objeción de todos los años: "¿y quién va a querer entrar a mi calculadora?". Nadie. Ese es exactamente el punto.

EJEMPLO

Las direcciones IPv4 son cuatro mil trescientos millones. Un equipo con conexión doméstica manda un millón de paquetes por segundo: barrer internet entera por un puerto es cosa de una hora, y repartido entre varias máquinas, de minutos. **Escanear internet no es una hazaña, es una tarea programada.** Y como el resultado se indexa y se vende —Shodan, Censys—, el atacante ni siquiera escanea: consulta.

Por eso el recuadro: un VPS empieza a recibir intentos de acceso dentro de la primera hora de existir.

✂ De la tabla miren Redis y Mongo: nacieron para una red interna de confianza y arrancaban sin contraseña. Cuando los contenedores volvieron trivial exponerlos, ese predeterminado permisivo se transformó en miles de bases secuestradas.

Y el daño casi nunca se queda en los datos de adentro. **A nadie le interesa su calculadora: les interesa su CPU, su ancho de banda y su IP limpia.**

Minado, que genera la factura. Correo no deseado, que deja la IP en listas negras. Ataques contra terceros, donde el responsable legal es el titular.

Y la cuarta, la más subestimada: **un servidor comprometido con claves privadas adentro no es un servidor comprometido, es todo lo que esas claves abren, comprometido.**

Apliquemos todo esto al caso que tienen más cerca. El panel recién instalado queda en el 3000 con los dos problemas juntos: la contraseña de administrador viaja en texto plano, y el puerto lo publicó Docker, así que ufw no lo protege.

La solución es un subdominio propio, y noten que **no mueve el panel**: el proceso sigue escuchando donde estaba, pero ahora hay un proxy inverso adelante que termina el cifrado y enruta por nombre. Mismo patrón que la aplicación entera en la Clase 4.

El orden es sagrado: primero verifican que el dominio nuevo entra, después cierran el 3000. Al revés, se quedan sin panel y sin forma de arreglarlo desde el panel.

Siete medidas, ninguna opcional en un entorno real. Las cuatro primeras ya las hicieron en clase.

La 7 merece un comentario porque es **la única que sigue trabajando cuando nadie mira.** La mayoría de los servidores comprometidos no cae por un ataque sofisticado sino por una vulnerabilidad conocida, publicada y ya parcheada. Entre que se publica y que se explota masivamente pasan horas: un servidor que nadie mira en toda la semana necesita esa automatización más, no menos.

✂ Y de fail2ban ubiquen qué aporta: no frena un ataque dirigido, encarece el automatizado, que es la inmensa mayoría del tráfico hostil.

Todo lo anterior fueron medidas. Esta diapositiva las explica, y es de 1975.

Mínimo privilegio: la base no publica puertos porque no necesita ser alcanzable. **Predeterminados seguros:** lo que no está permitido se niega — es la primera línea del firewall, y es lo que Redis y Mongo hacían al revés. **Economía del mecanismo:** "no publicar el puerto" es mejor que "publicarlo y filtrarlo" porque no hay regla que auditar ni herramienta en la que confiar.

Y abajo, defensa en profundidad: cuatro capas —clave, firewall, no publicar, fail2ban— y cada una supone que las otras pueden fallar.

Dos notas de actualización. Desde 2014 iptables tiene reemplazo, nftables, y Ubuntu 24.04 ya lo usa por debajo; para ustedes es invisible porque ufw se ocupa, pero importa al buscar documentación para no mezclar syntax.

Y Ed25519 en lugar de RSA: claves más cortas, firmas más rápidas y —la que importa— muchas menos formas de configurarlo mal.

Siete comprobaciones, y no son variantes de la misma: las cuatro primeras validan el acceso, las tres últimas el cierre.

Pero la que vale es la 5. Las otras seis describen intenciones; el nmap desde afuera describe hechos. Si devuelve algo más que 22, 80 y 443, tienen un puerto publicado por Docker y ya saben por qué.

Esta tabla no se la voy a leer: es de consulta y está en el apunte. Ténganla abierta durante la clase — las dos primeras filas son las que más pasan, y las dos que se arreglan por la consola de emergencia.

 Dirección: dejala en pantalla en silencio unos segundos para que se pueda pausar. No la narres.

Siete actividades. Dos son obligatorias y las quiero documentadas: la 1, inventario de puertos, y la 2, el contraste entre lo que dice ufw y lo que dice nmap desde otra máquina.

La 5 —cuántos PostgreSQL hay expuestos en Argentina— no está de relleno: es la que hace que todo esto deje de sonar a paranoia de profesor.

Cinco ideas. El VPS es una ilusión sostenida por un hipervisor, y de ahí salen el precio, la reinstalación en tres minutos y el vecino ruidoso. SSH autentica primero al servidor. El firewall no es ufw, es netfilter, y Docker se saltea ufw porque su tráfico va por FORWARD: la única verificación válida es escanear desde afuera. El descubrimiento es automático y no selectivo. Y los tres principios son de 1975.

El puerto más seguro es el que nunca se publicó. De ahí sale la red interna de la Clase 5.

El desarrollo está en el apunte. Nos vemos en la Clase 3, con Docker. Vengan con el nmap devolviendo tres puertos.

 Cierre: sostené la diapositiva dos o tres segundos después de terminar de hablar, antes de cortar.

Fin del video 3 y del capítulo. Es el que más diapositivas tiene y el más corto por diapositiva: las últimas cuatro son de consulta y se muestran, no se narran. Si venís pasado de tiempo, el recorte real es la 25 y la 28 — nunca la 17, que es la que justifica todo el capítulo.